

* ALGORITMI GREEDY

RECAP

- Problemi di ottimizzazione con sottostruttura ottima.
 - ad ogni passo fa la scelta LOCALMENTE OTTIMA
 - risolve UN SOLO SOTTOPROBLEMA: prima sceglie, poi calcola.

TOP-DOWN

SVANTAGGI: complessità, non sempre applicabile

• CARATTERISTICHE DEI PROBLEMI:

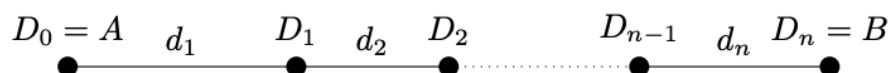
1. PROPRIETÀ DI SCELTA GREEDY: la soluzione ottima può essere costruita componendo scelte "ardite", si dimostra con "CUT & PASTE".

2. PROPRIETÀ DI SOTTOSTRUTTURA OTTIMA CON SPAZIO DEI SOTTO-PROBLEMI LINEARE: la soluzione ottima, che contiene la scelta greedy, contiene una soluzione all'UNICO SOTTOPROBLEMA che rimane dopo la scelta greedy.

• RICETTA PER LO SVILUPPO

1. Caratterizzare la STRUTTURA di SOLUZIONE OTTIMA.
2. Determinare la SCELTA GREEDY (che funzioni!).
3. Algoritmo TOP-DOWN → iterativo

Esercizio 33 Si supponga di voler viaggiare dalla città A alla città B con un'auto che ha un'autonomia pari a 2 km. Lungo il percorso si trovano $n - 1$ distributori $D_1, \dots, D_{n-1}$, a distanze di $d_1, \dots, d_n$ km ($d_i \leq 2$) come indicato in figura



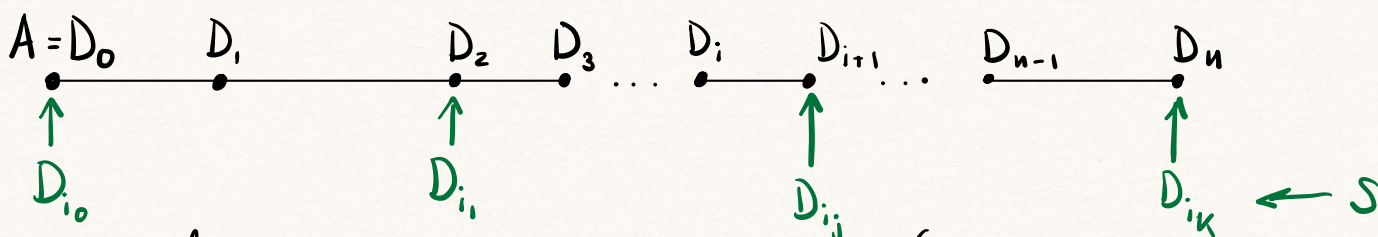
L'auto ha inizialmente il serbatoio pieno e l'obiettivo è quello di percorrere il viaggio da A a B , minimizzando il numero di soste ai distributori per il rifornimento.

- Introdurre la nozione di soluzione per il problema e di costo della una soluzione. Mostrare che vale la proprietà della sottostruttura ottima e individuare una scelta che gode della proprietà della scelta greedy.
- Sulla base della scelta greedy individuata al passo precedente, fornire un algoritmo greedy $\text{stop}(d, n)$ che dato in input l'array delle distanze $d[1..n]$ restituisce una soluzione ottima.
- Valutare la complessità dell'algoritmo.

SOLUZIONE:

i. SOLUZIONE: la specifica è una sequenza $A = D_0, \dots, D_i, \dots, D_n = B$ di soste possibili.

Definiamo la distanza tra due soste $d_{ij} = \sum_{k=i+1}^j d_k$.



$\Rightarrow$ una soluzione è una sequenza $S = (A = D_{i_0}, D_{i_1}, \dots, D_{i_k} = B)$

Tale che $d_{i_j, i_{j+1}} \leq 2 \quad \forall j \in \{0, k-1\}$, con $i_0 < i_1 < \dots < i_k$.

Il costo $c(S) = k - 1$, n. di soste.

SOTTOSTRUTTURA OTTIMA: sia $S = D_{i_0} \dots D_{i_k}$ soluzione ottima per $D_0 \dots D_n$. Considera il sottoproblema $D_{i_1} \dots D_n$, allora $S_1 = D_{i_1} \dots D_{i_k}$ è soluzione ottima: è una soluzione, e se ce ne fosse una migliore S'_1 tale che $c(S'_1) < c(S_1)$, allora $S' = D_{i_0} S'_1$ sarebbe soluzione migliore per $D_0 \dots D_n$, ovvero $c(S') = c(S'_1) + 1 < c(S_1) + 1 = c(S)$, contraddicendo l'ipotesi

che S' fosse soluzione ottima.

SCELTA GREEDY:

- $i_0 = 0$

- scelgo la sosta più lontana possibile (entro l'autonomia d), definisco $i_0 = \max \{j \mid d_{0,j} \leq d\}$.

$\Rightarrow$ data una soluzione ottima $S = D_{j_0} \dots D_{j_k}$, $j_0 = i_0 = 0$ e $j_1 \leq i_1$ (massimo) $\Rightarrow S' = D_{j_0} \textcircled{D_{i_1}} \dots D_{j_k}$ è soluzione ottima per il problema, con $c(S') = c(S)$ (non può essere $c(S') < c(S)$, perché contraddirebbe S soluzione ottima).

ii. STOP(d, n, a): // $d[1, \dots, n]$ array delle distanze, a autonomia
 $S[1 \dots n] = \text{zeros}(n)$ // array di soluzioni, metto a 1 i dist. a cui mi fermo
 $S[1] = 1$
 $\text{dist} = d[1]$
 for $i = 2$ to n :
 if $\text{dist} + d[i] > a$:
 $S[i-1] = 1$
 $\text{dist} = d[i-1]$

iii. COMPLESSITÀ: $O(n)$, poiché scorro linearmente $d[1 \dots n]$



Esercizio 34 L'ufficio postale offre un servizio di ritiro pacchi in sede su prenotazione. Il destinatario, avvisato della presenza del pacco, deve comunicare l'orario preciso al quale si recherà allo sportello. Sapendo che gli impiegati dedicano a questa mansione turni di un'ora, con inizio in un momento qualsiasi, si chiede di scrivere un algoritmo che individui l'insieme minimo di turni di un'ora sufficienti a soddisfare tutte le richieste. Più in dettaglio, data una sequenza $\vec{r} = r_1, \dots, r_n$ di richieste, dove r_i è l'orario della i -ma prenotazione, si vuole determinare una sequenza di turni $\vec{t} = t_1, \dots, t_k$, con t_j orario di inizio del j -mo turno, che abbia dimensione minima e tale che i turni coprano tutte le richieste.

- Formalizzare la nozione di soluzione per il problema e il relativo costo. Mostrare che vale la proprietà della sottostruttura ottima e individuare una scelta che gode della proprietà della scelta greedy.
- Sulla base della scelta greedy individuata al passo precedente, fornire un algoritmo greedy `time(R, n)` che dato in input l'array delle richieste `r[1..n]` restituisce una soluzione ottima.
- Valutare la complessità dell'algoritmo.

SOLUZIONE:

[Assumo $R = r_1, \dots, r_n$ ordinate in modo crescente].

- i. SOLUZIONE: una soluzione è una sequenza di turni $T = t_1, \dots, t_k$ tale che $\forall i = 1 \dots n \exists j = 1 \dots k$ tale che $r_i \in [t_j, t_j + 59]$ (intervallo di un'ora che inizia a t_j), con $t_j < t_l$ se $j < l$.

SOTTOSTRUTTURA OTTIMA: se $T = t_1, t_2, \dots, t_k$ è soluzione ottima per le richieste $R = r_1, \dots, r_n$, allora $T_1 = t_2, \dots, t_k$ è ottima per il sottoproblema $R' = r_1, \dots, r_n$ tali che $r_1, \dots, r_{l-1} \in [t_1, t_1 + 59]$ e $r_l \notin [t_1, t_1 + 59]$. Se u fosse una soluzione migliore T'_1 con costo $s < k - 1 \Rightarrow T' = t_1 T'_1$ avrebbe costo $s + 1 < k$, contraddicendo l'ottimalità di T .

SCELTA GREEDY: $t_1 = r_1$: ogni soluzione ottima contiene questa soluzione: sia $T' = t'_1, \dots, t'_k \Rightarrow t'_1 \leq r_1$, dato che la prima richiesta deve essere servita, e $T = \textcircled{t_1} t'_2, \dots, t'_k$

è ancora soluzione ottima (soddisfa tutte le richieste di T').

ii. $\text{time}(R, n)$ // $R[1 \dots n]$ array delle richieste

$t[1] = R[1]$

$\text{turni} = 1$

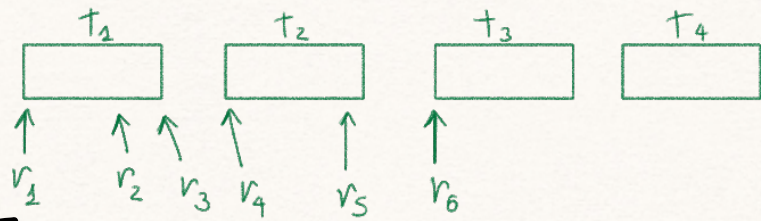
for $i = 2$ to n :

if $t[\text{turni}] + 60 \text{ min} < R[i]$:

$\text{turni}++$

$t[\text{turni}] = R[i]$

return t



iii. COMPLESSITÀ: $O(n)$, poiché scorro linearmente $R[1 \dots n]$

